Meridian administrator training

For the customer's Meridian administrator and their SAP Basis colleague: install and first start, licence, users and access, connecting SAP, extraction, scoring and money, fixing data and write-back, notifications, operations and troubleshooting.

Edition of 31 October 2026.

Contents

1. Module 1: Install and first start
2. Module 2: Licence
3. Module 3: Users and access
4. Module 4: Connecting SAP
5. Module 5: Extraction, runs and schedules
6. Module 6: Scoring and money
7. Module 7: Fixing data and write-back
8. Module 8: Notifications and integrations
9. Module 9: Operations
10. Module 10: Troubleshooting
11. Module 11: Prove the numbers

About this course

A course for the customer's Meridian administrator and the SAP Basis colleague who connects Meridian to SAP. It takes you from an empty Linux host to a deployment that is installed over HTTPS, licensed, connected to SAP with a read-only role, extracting on a schedule, scoring, routing fixes and backed up.

Work through the modules in order. Each one has learning objectives, steps, checks ("you should now see …") and a short exercise. Every setting, route and command named here exists in the release this edition describes. The screenshots are taken from the product's demo build with sample data (the "Demo data" bar at the top of each one); your own deployment shows your data and never shows that bar. They show the light theme, the default; each person can switch to dark under Preferences, Display, or from the account menu.

Every screen opens the same way: a banner with the verdict for what you are looking at (go, at risk or no-go), the figures behind it and one primary action, then the detail. Aletheia, the grounded assistant, is docked beside the canvas on every screen; it answers from the run you are looking at and cites the rules behind each number.

Who does what:

- **Meridian administrator**: modules 1 to 3 and 6 to 11.
- **SAP Basis**: modules 4 and 5, and the SAP side of modules 7 and 11.

Module 1: Install and first start

Learning objectives

- Check the host against the prerequisites before installing.
- Run the installer, choose an HTTPS mode and know what each one does.
- Sign in for the first time and change the administrator password.
- Read the health endpoints and the health checks.

Prerequisites

- A Linux host with Docker 24 or later, `curl`, `python3` and `openssl`, run as root (`sudo`).
- Outbound HTTPS to the licence server (online licence) or an offline licence token (air-gapped).
- A DNS name for the host if you want a Let's Encrypt certificate.
- Network access from the host to the SAP application servers (RFC) or the SAP cloud APIs.
- `gpg` for encrypted backups (module 9).

`scripts/preflight.sh` checks the host before you start.

Steps

1. Run the installer: `sudo bash scripts/meridian-deploy.sh`. It asks for the server name, the LLM tier, the SSL mode, and the first administrator (`ADMIN_EMAIL`, `ADMIN_NAME`, `ADMIN_PASSWORD`), writes `.env` with fresh secrets, migrates the database, creates the administrator with `scripts/manage_users.py` and starts the stack. For unattended installs pass `--non-interactive` with a filled `.env`; for private registries or air-gapped hosts see `--image-source registry` and `--image-source local --image-tarball <path>`.
2. Choose the **SSL mode**:
   - `1`: no TLS, HTTP only. For a closed test network only.
   - `2`: a self-signed certificate. The installer enables HTTPS in nginx with `scripts/nginx-https.sh`: port 443 serves Meridian and port 80 answers every request except the health probes with a redirect to HTTPS.
   - `3`: a Let's Encrypt certificate for the server name, enabled the same way.
3. Open the address the installer prints and sign in with the administrator email and password.
4. **Change the password.** Meridian asks for a new password before anything else when an account carries the "must change password" flag: the seeded development account (`admin@meridian.local`), and any account an administrator has reset. The new password must have at least 12 characters, must not be the email name, a common password or the current one.
5. Read the health:
   - `https://<server>/health` is the readiness probe. It answers `200` with `status: ok` only when Postgres and Redis answer, and `503` with `status: degraded` and the state of each component otherwise. It also reports the version, the model provider, the cached licence and the email backend.

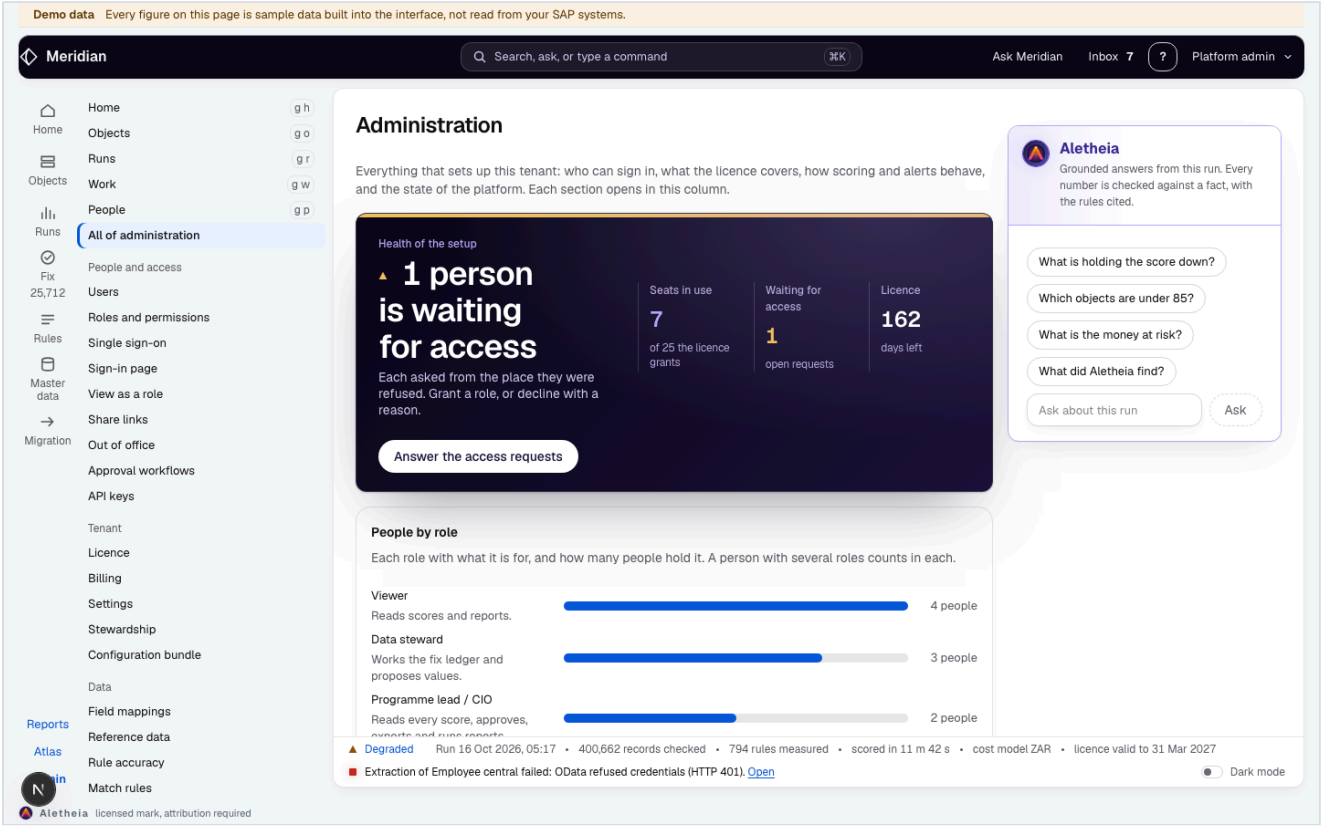
- `/health/live` is the liveness probe inside the stack: it answers whenever the API process runs, without touching Postgres or Redis, so a restart policy never cycles the API for a short database outage. nginx does not publish it; read it with `docker compose exec api curl -s http://localhost:8000/health/live`.
- The installer writes `/opt/meridian/healthcheck.sh`, which checks each container in turn.
- In the product: **Administration, Platform and updates, Health checks** (database, cache and queue, background workers, licence, email delivery, file storage), refreshed every 30 seconds.

`scripts/update.sh` re-applies HTTPS after every update when a certificate is present, so an update never falls back to HTTP.

The interactive API documentation (`/docs`, `/redoc`, `/openapi.json`) is not served on a production deployment: it answers only when `MERIDIAN_ENV=development` or `MERIDIAN_OPENAPI_PUBLIC=1` is set, because nginx proxies those addresses without sign-in.

Checks

- You should now see the sign-in page over `https://` (SSL mode 2 or 3), and `http://<server>/` redirect to it.
- After the password change you should land on the Landscape with **Administration** in your account menu.
- `curl -s https://<server>/health` should print `"status": "ok"`.



Administration: the health of the setup with its one action, people by role, and every section, grouped as People and access, Tenant, Data, Notifications and Operations.

Exercise

Stop Redis for a minute (`docker compose stop redis`), call `/health` and note the `503` and which component reads `down`; check that `/health/live` still answers; start Redis again (`docker compose start redis`).

Module 2: Licence

Learning objectives

- Know how online and offline licences are validated, and what they unlock.
- Understand the Ed25519 transition window and when to turn it off.
- Know what the 14-day offline grace does and what users see before and after it.

How a licence is checked

Every licence path is verified against the vendor's Ed25519 public key compiled into the image (`api/middleware/licence_keys.py`): the online validation response, the offline token and the offline file alike. There is no key to set in `.env`, and an unsigned or wrongly signed licence grants nothing.

- **Online** (`MERIDIAN_LICENCE_MODE=online`, the default): the API sends `MERIDIAN_LICENCE_KEY` to `MERIDIAN_LICENCE_SERVER_URL` and caches a valid answer for 6 hours.
- **Offline** (`MERIDIAN_LICENCE_MODE=offline`): `MERIDIAN_LICENCE_TOKEN` holds a signed token issued by Meridian HQ, for air-gapped hosts. It never phones home.

The transition window

`MERIDIAN_LICENCE_ALLOW_LEGACY_SIGNATURES` is `true` by default in this release. While it is on, Meridian also accepts the older signatures (an HMAC-signed offline file, an RS256 offline token, an RSA-signed online response and, with no RSA key configured, an unsigned online response), and logs each acceptance. It exists because the licence worker at Meridian HQ is deployed separately: an installation that updated before the worker signs with the vendor key would otherwise refuse its own licence.

Turn it off (`MERIDIAN_LICENCE_ALLOW_LEGACY_SIGNATURES=false` in `.env`, then restart the API) once Meridian HQ confirms the licence worker signs with the vendor key. A later release makes `false` the default.

The offline grace

When the licence server cannot be reached, the last valid signed response is honoured for `MERIDIAN_LICENCE_OFFLINE_GRACE_DAYS` (default 14, at most 30) days after it was signed. It is kept on disk at `MERIDIAN_LICENCE_STATE_PATH` (default `/var/lib/meridian/licence-last-valid.json`) and re-verified when it is loaded, so a restart keeps the grace and editing the file grants nothing.

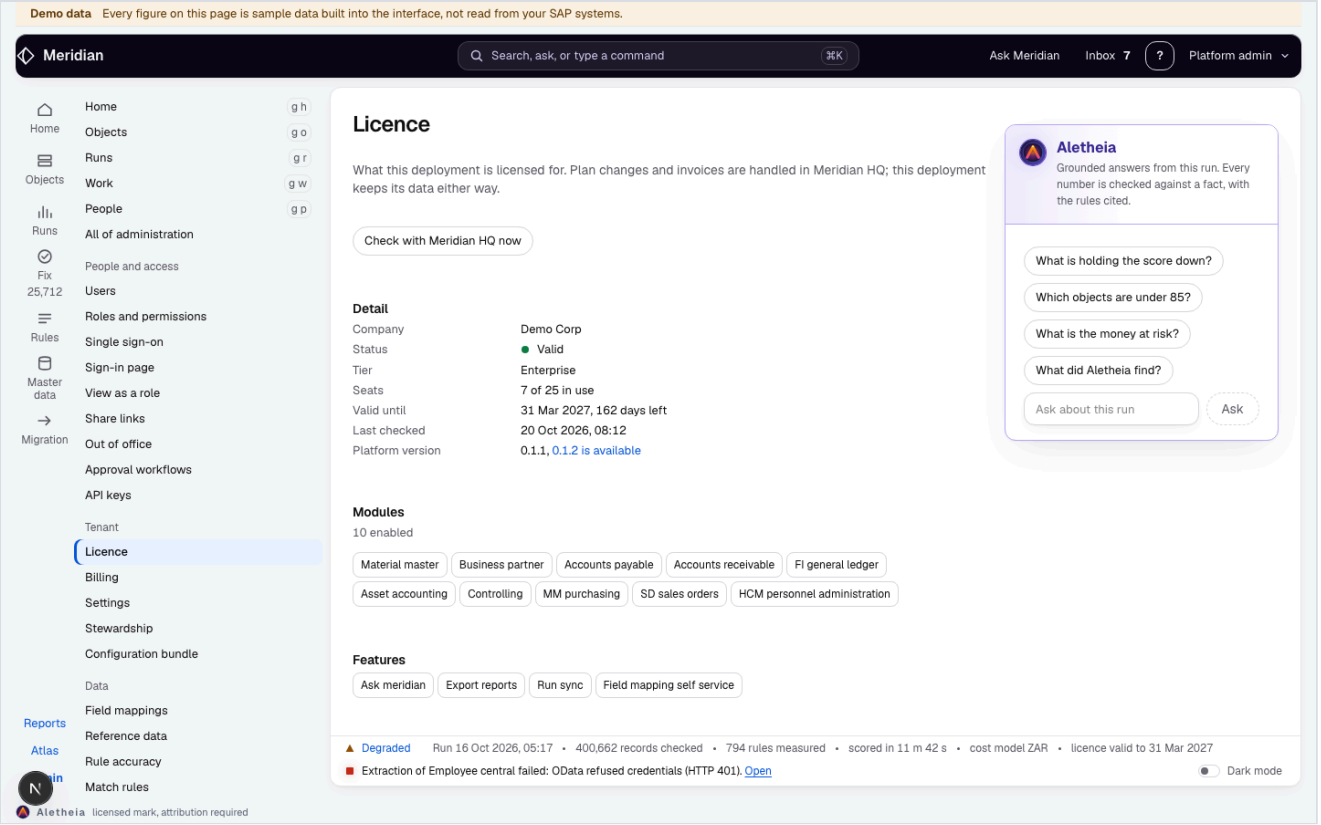
What people see:

- **During the grace** everything keeps working. `GET /api/v1/licence` reports status `offline_grace` with when the response was signed, when the grace ends and the days left; every gated API answer carries the header `X-Licence-Warning: offline_grace`, and the API log warns. The Licence page in Administration still reads **Valid**, so watch the status and the log rather than the screen.
- **After the grace** gated routes answer `403` with `licence_server_cutoff`, and people are sent to the "Licence needed" page: "Meridian HQ could not be reached to validate the licence".

Steps

1. Open **Administration, Licence**. Read the company, status, tier, seats, validity, last checked, platform version and the licensed modules and features.

2. Press **Check with Meridian HQ now** to validate at once instead of waiting for the cache.



Administration, Licence: status, seats, validity and the licensed modules.

Checks

- You should now see **Valid** and the expiry date with the days left.
- `curl -s https://<server>/health` shows the same licence in its `licence` field.

Exercise

Find in `.env` which licence mode this deployment uses, and write down the date by which you will turn the transition window off. Then answer: if the licence server is unreachable from Friday evening, until when does Meridian keep working?

Module 3: Users and access

Learning objectives

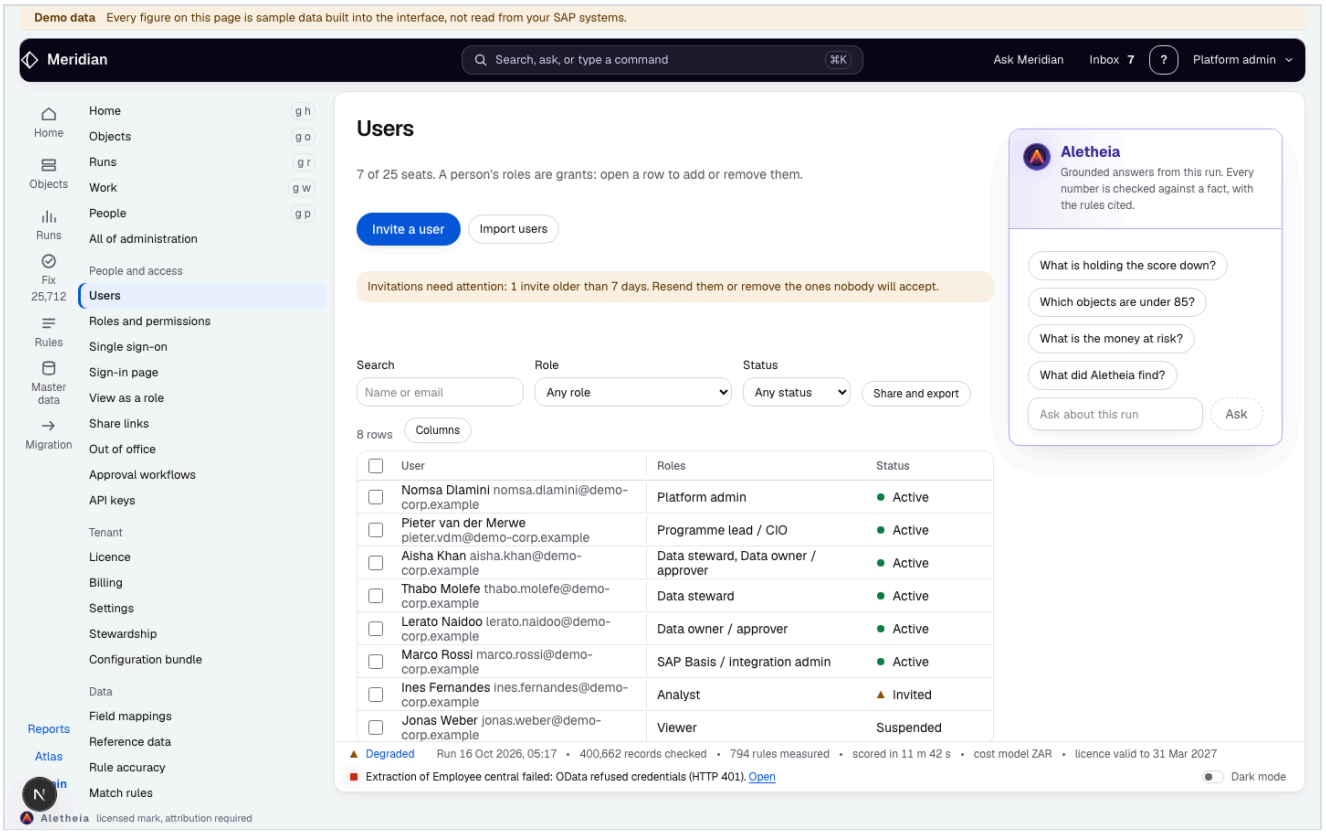
- Invite people and give them roles, scoped where needed.
- Connect single sign-on and decide whether to require it.
- Provision people with SCIM and map identity provider groups to roles.
- Issue API keys, and read the sign-in events in the audit log.

Roles

A person's roles are grants. Built-in roles cannot be changed; copy one to make a custom role. A grant can be scoped to one system or object, and can expire. The roles include platform admin, programme lead, data steward, data owner, SAP Basis, process analyst, analyst, auditor and viewer; each has its own start screen, and the account button names the person's role. **Administration, Roles and permissions** shows the matrix "What each role may do".

Steps

1. **Administration, Users**: **Invite a user** with an email and roles, or **Import users** from a CSV with a preview. The seat meter counts active and invited users against the licence. Invitations expire after 7 days; resend or copy the link from the person's row.
2. **Administration, Single sign-on**: OpenID Connect with your identity provider (Microsoft Entra ID, Okta). Enter the issuer address, client ID and client secret ("Stored encrypted and never shown again."), register the redirect address shown there at the provider, then **Test sign-in**. People who sign in for the first time get the Viewer role and no access grants.
3. **Administration, Sign-in page**: the allowed email domains and **Require single sign-on**. Password sign-in then stops for everyone except administrators, who keep it as a way in if the provider is down. Ten failed sign-ins lock an account for 15 minutes.
4. **Administration, Users, Provisioning (SCIM)**: **Create a token**, then give your identity provider the SCIM base URL (`/scim/v2`) and the token. Under **Group to role**, map identity provider groups to Meridian roles; saving re-syncs everyone in a SCIM group. **Rotate the token** stops the old one at once; **Revoke** stops provisioning (people keep their accounts).
5. **Administration, API keys**: a key acts as its own service account with only the roles you give it, and shows in the audit log under its name. The key is shown once, when you create it.
6. **Administration, Audit log**: every change, by whom and when, with filters by person, action, object kind and dates, and **Export CSV** and **Export Excel**. Sign-in events are recorded too: `auth.login`, `auth.login_failed` (the email as typed and the reason, never the password), `auth.lockout`, `auth.logout`, `auth.password_change`, and for single sign-on `auth.sso_login`, `auth.sso_refused` and `auth.sso_user_created`.

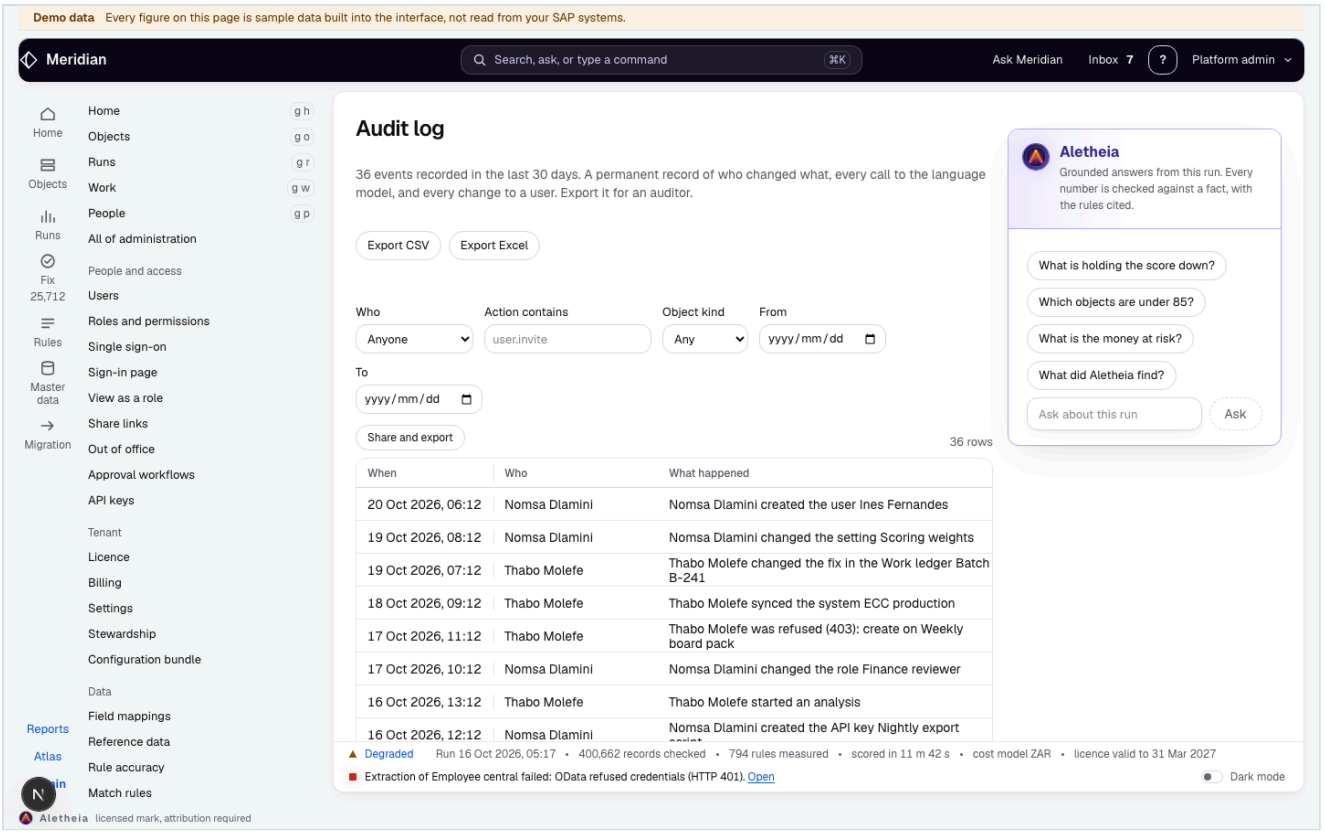


Administration, Users: seats, roles as grants, invitations and the SCIM card.

Signed-in requests run in the tenant named by the verified token, never a fixed one. `MERIDIAN_TENANT_ID` sets this deployment's own tenant, used for sign-in, the pages before sign-in, API keys and the HQ manifest; a single-tenant installation keeps the default.

Checks

- You should now see the invited person with status **Invited**, and after they accept, **Active**.
- In the audit log, filtering on the action `auth.login` should list your own sign-in.



Administration, Audit log: filters, exports and the AI calls view.

Exercise

Invite a colleague as a viewer, scope a data steward grant to Material master only, sign in with a wrong password three times from a test account, and find the three `auth.login_failed` rows in the audit log.

Module 4: Connecting SAP

Learning objectives

- Choose the system type and fill in the connection.
- Create the read-only SAP role from the template.
- Choose the extraction path (table reader, ODP or OData) and the table reader your Basis team accepts.
- Test the connection and read connection health; connect a cloud system.

System types

ECC, S/4HANA on premise, Extended Warehouse Management (RFC, user and password), S/4HANA Cloud (OData V4, OAuth 2.0), SAP BTP (OData, XSUAA OAuth 2.0), SuccessFactors (OData V2, basic or OAuth 2.0), Concur (REST, OAuth 2.0) and Ariba (REST, OAuth 2.0 and an API key).

The read-only role (SAP Basis)

`docs/sap/authorisation-role-template.md` is the PFCG role for the RFC user: copy it into a role such as `Z_MERIDIAN_READ` and assign it to a system or communication user (SU01 type B or C). It grants:

- `S_RFC` (`ACTVT = 16`) on the named function modules only: `RFC_PING`, `RFC_SYSTEM_INFO`, `DDIF_FIELDINFO_GET`, `RFC_GET_FUNCTION_INTERFACE`, `RFC_METADATA_GET`, optionally `EM_GET_NUMBER_OF_ENTRIES`, and one table reader.
- `S_TABU_NAM` (preferred) or `S_TABU_DIS` with `ACTVT = 03` on the tables of the modules you extract, the data dictionary, organisational structure and, for delta extraction, `CDHDR` and `CDPOS`.
- No transaction codes, no `S_DEVELOP` and no change activity. With this role Meridian cannot change SAP data.

For the ODP path the role adds `S_RFC` for function group `RODPS_REPL` and the ODP authorisations (`S_RO_OSOA`, and the CDS views' access controls on S/4HANA); for the OData path, `S_SERVICE` for the Gateway services and the back-end display authorisations of those APIs. The template marks the entries to confirm in SU24 or SE37.

Choosing the extraction path

Many Basis teams refuse `RFC_READ_TABLE` in production. Each ABAP system's connection settings have an **Extraction path**:

Choice	What it does
Table reader (default)	Reads every table with the table reader below.
ODP, table reader for the rest	Reads the tables an ODP provider covers (`0MATERIAL_ATTR` for MARA, `0MAT_PLANT_ATTR` for MARC, `0CUSTOMER_ATTR` for KNA1, `0VENDOR_ATTR` for LFA1, `0GL_ACCOUNT_ATTR` for SKA1, and on S/4HANA the CDS views `I_Product`, `I_Customer`, `I_Supplier`, `I_BusinessPartner`) through the `RODPS_REPL_ODP_*` functions. A field the provider does not carry is listed for the table and its rules are not evaluable. Other tables use the table reader.

Choice	What it does
Automatic: table reader	The table reader for every table. ODP over RFC is used only when you choose ODP explicitly, after confirming with SAP that your licence allows it (SAP Note 3255746).
OData through the SAP Gateway	S/4HANA on premise only. Reads the released APIs your Gateway exposes (`API_PRODUCT_SRV`, `API_BUSINESS_PARTNER`, `API_GLACCOUNTINCHARTOFACCOUNTS_SRV`) with this system's user, sending its client as `sap-client`. Enter the **Gateway base URL**.

A table read with a date window, a scope or by parent keys always uses the table reader, because ODP reads a provider in full. The job's **Tables** list has a **Read through** column, and the Basis home shows each system's extraction path under **Connections** and, under **Last extraction**, how its tables were read.

Before choosing ODP for production, read SAP Note 3255746 with your SAP account team: it restricts the use of the ODP RFC modules by third-party applications. Where it applies, choose OData or the table reader.

Choosing the table reader

Each ABAP system's connection settings have a **Table reader**; **Deployment default** uses `MERIDIAN_RFC_READ_FUNCTION` in `.env`:

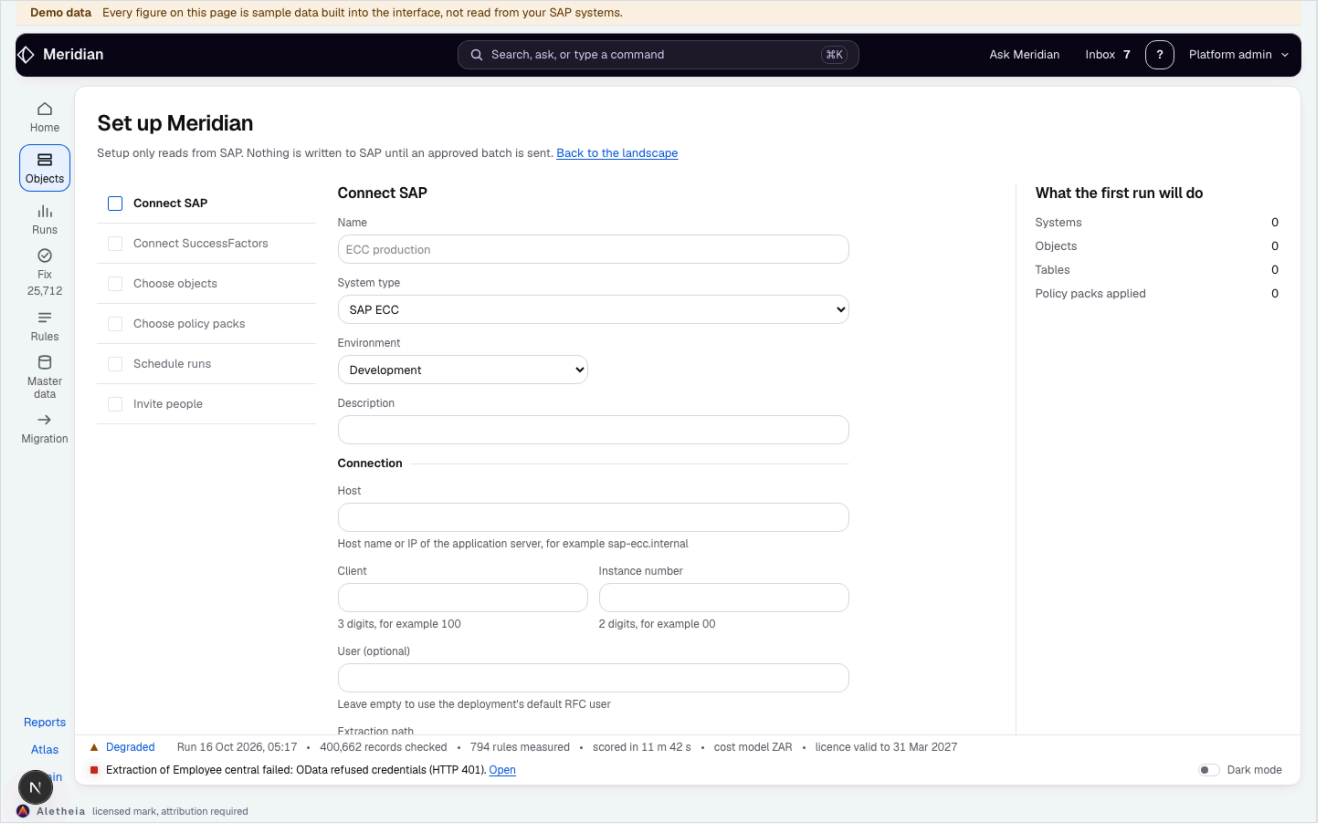
Value	Reader
`RFC_READ_TABLE` (default)	512-byte rows; wide tables are read in column groups and joined on the key
`/BODS/RFC_READ_TABLE2`	the Data Services reader, rows up to 30,000 bytes; wide tables in one pass
`BBP_RFC_READ_TABLE`	the SRM reader, the same interface as `RFC_READ_TABLE`; some sites allow it when they refuse `RFC_READ_TABLE`
`auto`	probes `RFC_FUNCTION_SEARCH` for `/BODS/RFC_READ_TABLE2`, then `BBP_RFC_READ_TABLE`, else `RFC_READ_TABLE`

The extracted data is the same whichever reader or path is used. `RFC_FUNCTION_SEARCH` is needed for `auto` (reader or path).

Steps

1. Open **Objects** on the rail and choose **Connect a system** beside Systems in its menu, or go straight to `/s/new`. **Systems** (`/s`) lists every connected system with its connection health and last sync. **Set up Meridian** runs in six steps: **Connect SAP**, **Connect SuccessFactors** (optional; **Skip for now**), **Choose objects**, **Choose policy packs**, **Schedule runs** and **Invite people**, with **What the first run will do** beside them. In **Connect SAP**, name the system, choose the **System type** and the **Environment**, then for RFC systems **Host**, **Client** (3 digits), **Instance number** (2 digits) and an optional **User** (empty uses the deployment's default RFC user, `SAP_RFC_USER`), with the password under **Sign-in secrets**, then the **Extraction path** and the **Table reader**. Cloud systems ask for their base URL and OAuth details instead.
2. **Test connection**. A passing test reads like "RFC connection succeeded (client 100, 41 ms)".

3. **Connect and continue**, then work through the remaining steps and **Finish setup**. Nothing is read from SAP until the first run; setup only reads, and nothing is written to SAP until an approved batch is sent.
4. If SAP host names resolve only on your corporate DNS, set `MERIDIAN_DNS_1` (and `MERIDIAN_DNS_2`) in `.env`; `scripts/dns-overlay.sh` gives the API and worker those resolvers, and the installer and `update.sh` apply it. Without them containers use the host's resolver.



Connect a system: system type, environment, connection and the test.

Checks

- You should now see the system in the **Objects** menu and on **Systems** (`/s`) and, for the SAP Basis role, a row under **Connections** on the Basis home with "Connected" and what the last test said.
- A table the user may not read shows "No authorisation to read this table. The user needs read access to it."; SU53 on the SAP side shows the failed check.

Exercise

With your Basis colleague, generate the role from the template on a quality system, set the system's **Table reader** to **Find the first available** and its **Extraction path** to **Automatic**, test the connection, run an extraction of Material master, and note in the job inspector's **Read through** column which tables used ODP and which the table reader, and why (the extraction log lists the fields ODP could not deliver).

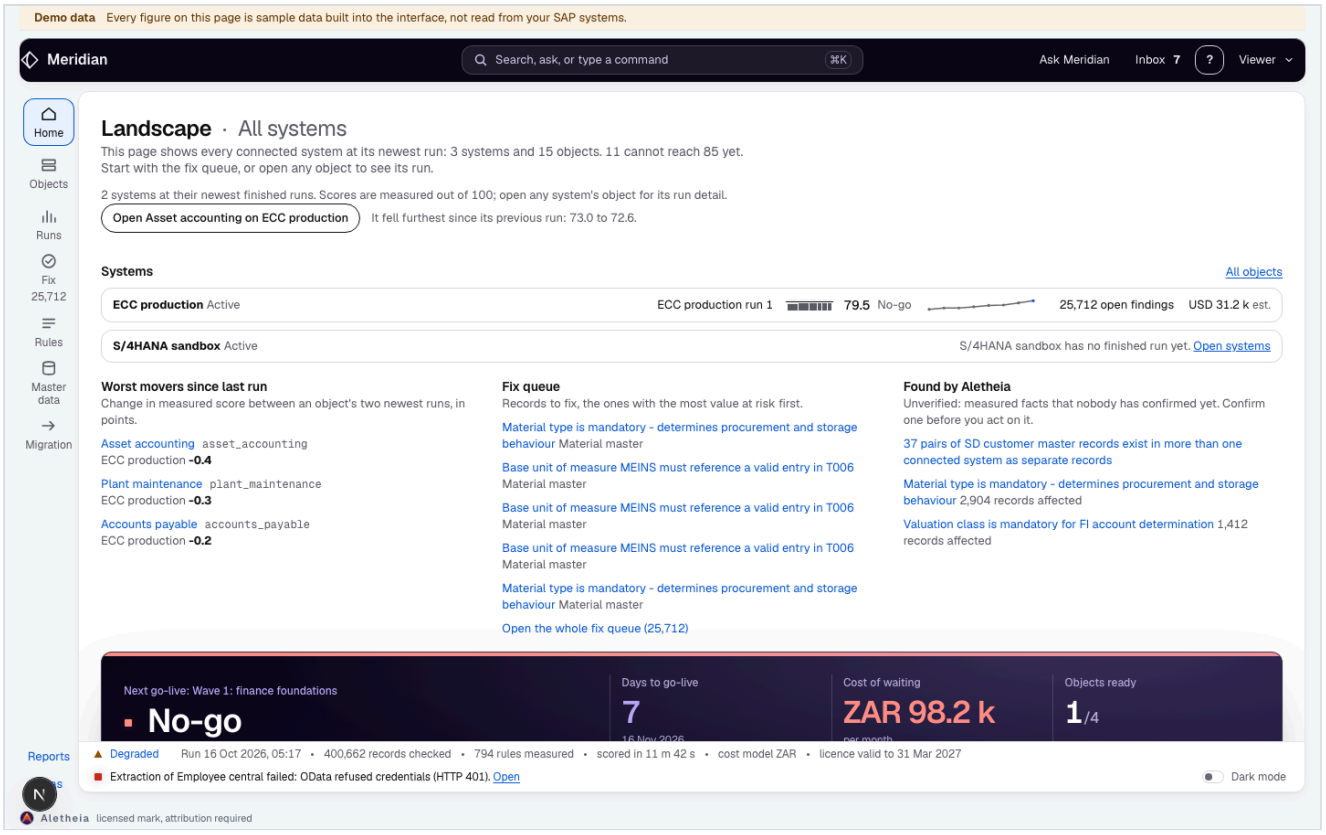
Module 5: Extraction, runs and schedules

Learning objectives

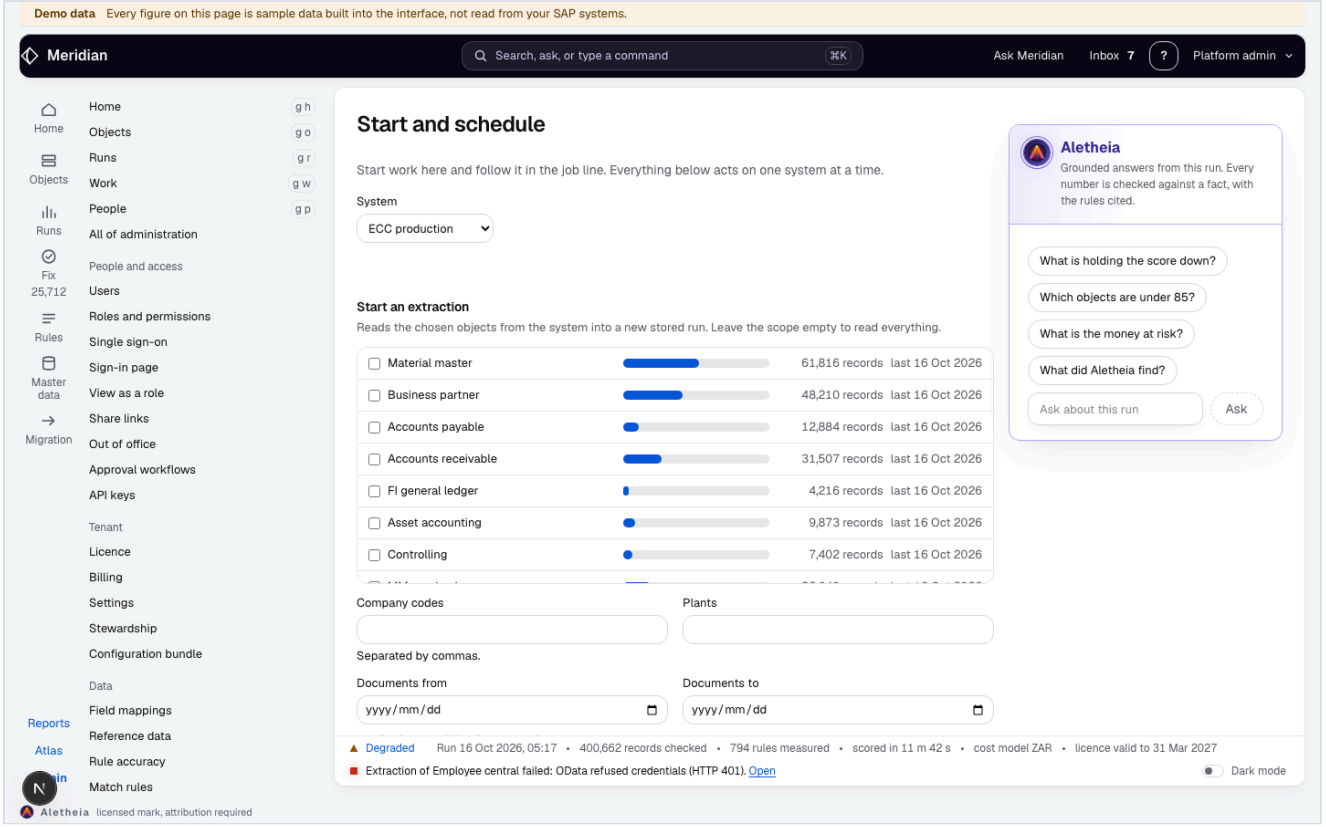
- Start and schedule extractions and analyses.
- Use dry runs and delta extraction.
- Read the jobs view and triage failed jobs from the Basis home.

Steps

1. **Dry run**: in the system inspector choose the objects and press **Dry run**. It shows what an extraction would read, without reading it: rows per table from the last extraction and an estimated duration.
2. **Administration, Start and schedule** (the runs themselves are on **Runs**): **Start an extraction** with the objects, company codes, plants, a document date window and **Analyse straight after extracting**; **Changes only since last full download** reads only records changed since the last full download (delta). A delta runs as a full extraction when the last full one is older than `MERIDIAN_DELTA_FULL_DAYS` (default 7). **Start an analysis** scores a stored extraction; **Load configuration** reads the configuration tables; **Upload a file** turns a CSV, TSV, Excel, JSON or Parquet file of one object into a run.
3. Every start form shows its prediction first: duration band, records, cost (from the cost model's run cost) and queue position.
4. **Scheduled runs**: **Schedule a run** with a rhythm and **Changes** or **Everything**. Times are UTC. Due runs start within 5 minutes of their time.
5. **Runs, Jobs**: every extraction, analysis and configuration load with its state. Open a failed job for its stages, the error in plain words and **Retry**.
6. **The Basis home**: SAP Basis lands on the landscape; below the go-live banner and the landscape map sits what they look after: **Connections** (health and what the last test said), **Last extraction** per system with records and state, **Failed jobs to triage** (a failed job not followed by a success of the same kind), **Authorisation errors** (refused credentials and missing authorisations from the tests and jobs) and **Schedules** with the next run and the mode.



The Basis home: connections, last extractions, failed jobs, authorisation errors and schedules.



Administration, Start and schedule: start an extraction or an analysis with its prediction, and the schedules.

Checks

- You should now see the run on **Runs** with its records, rules, score range and status.

- A failed extraction should appear once under **Failed jobs to triage** and disappear after a successful retry.

Exercise

Schedule Material master on Mondays at 04:00 UTC as **Changes**, then dry run it and compare the predicted duration with the actual one after Monday's run.

Module 6: Scoring and money

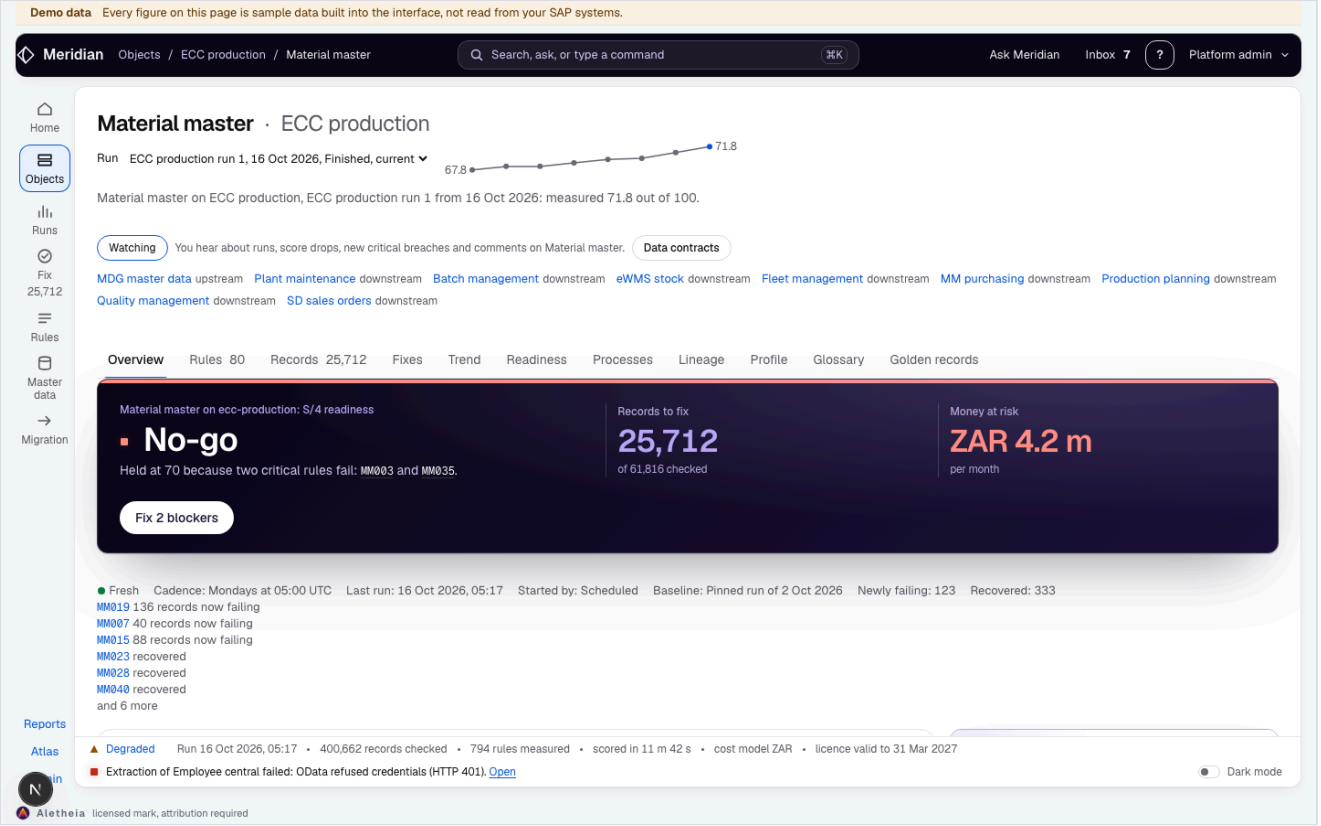
Learning objectives

- Read the score formula, the cap and the measured score.
- Set the cost model and know why money is an estimate until it is saved.
- Apply a policy pack.

The score

FENCE0 Each dimension score averages the record-level pass rates of its rules. The cap rule is fixed: one critical rule over tolerance caps the score at 85, two or more at 70. A critical rule is over tolerance when it fails at least the share or the number of records set under **Administration, Settings, Cap rule and tolerance** (by default 10% of rows or 25 rows).

Where a cap applies, the score reads with both numbers, for example "70.0 capped, 71.8 measured", and the receipt says "Measured 71.8, the tick on the ruler. Capped at 70 by MM003 and MM035, two critical rules over tolerance." The landscape and the Objects board rank objects by the measured score, so capped objects still sort by how good they are.



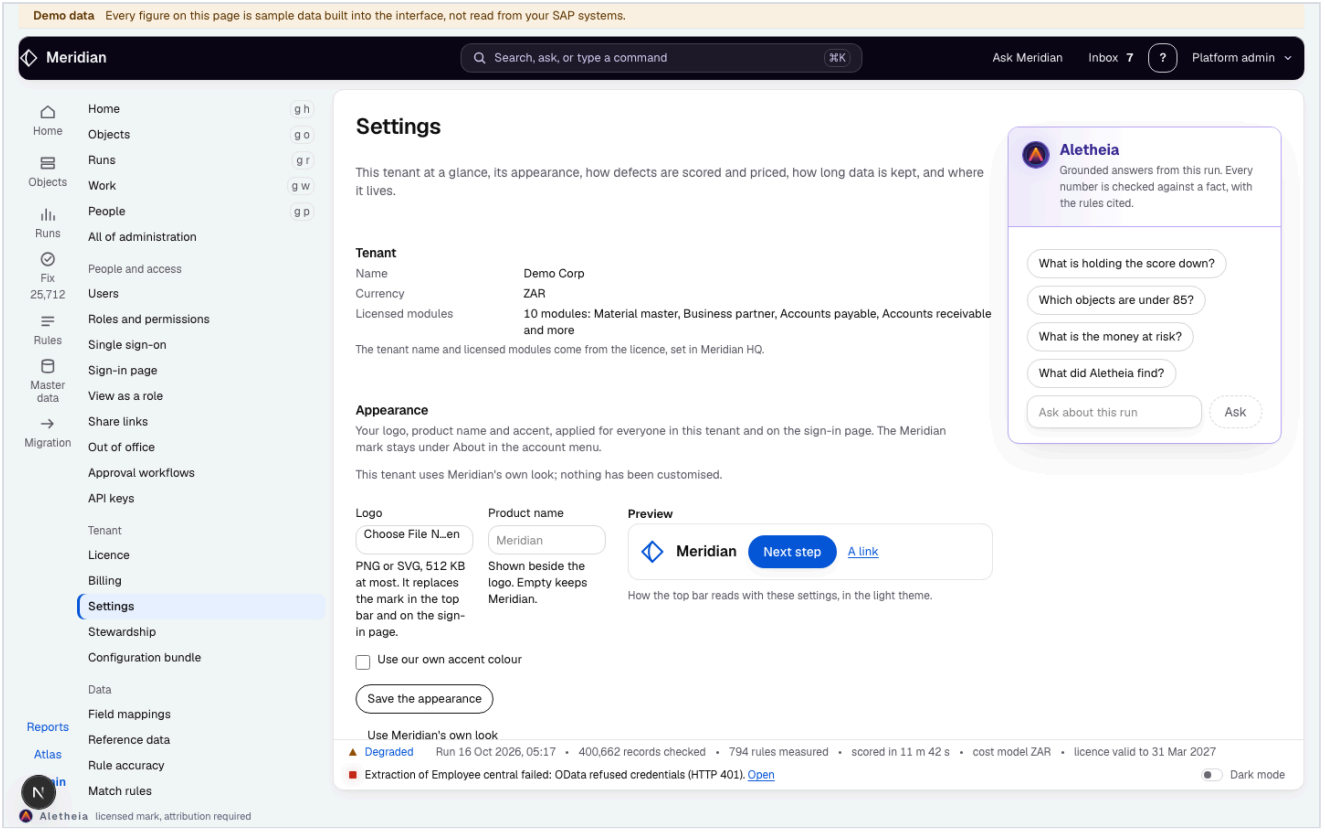
The object receipt: measured, capped, by which critical rules.

Money is an estimate

Every amount (value at risk, ROI, planner figures) comes from the tenant's cost model: the cost of one failing record by severity. Every amount carries the marker **est.**. Until the tenant saves its own cost model, the figures use the shipped starting assumptions: the marker reads **default estimate** and the note says "an estimate from default assumptions; set your cost model in Administration".

Steps

1. **Administration, Settings**: check **Dimension weights** (they must add up to 100; **Use the DAMA defaults**), **Score bands**, **Cap rule and tolerance**.
2. **Cost model and currency**: set the currency and the cost of one blocked record by severity, then **Save cost model**. The run cost (per worker hour and per million records) feeds the run prediction.
3. **Administration, Policy packs**: GxP, SOX and GDPR. **Apply the pack** marks failing critical rules of its classes as policy breaches on the landscape, the ruler and the receipt.



Administration, Settings: weights, bands, cap rule and tolerance, cost model.

Checks

- You should now see amounts marked **est.** instead of **default estimate**.
- A change to weights or tolerance applies to the next analysis; scores stored with earlier runs stay as they were.

Exercise

Before saving the cost model, find one amount marked **default estimate** on Work. Save your cost model, reload and check the marker reads **est.**

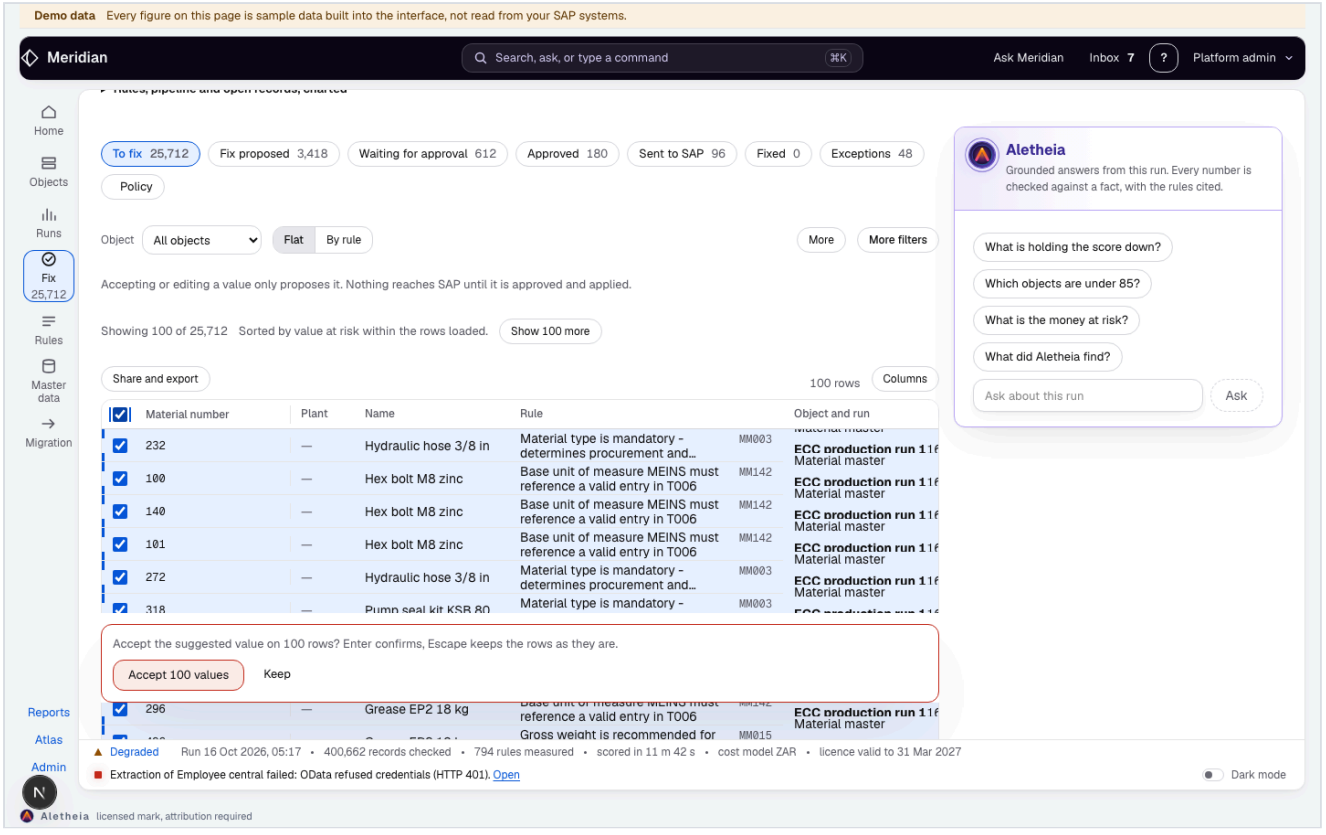
Module 7: Fixing data and write-back

Learning objectives

- Know how fixes flow from the Work ledger to SAP, with four eyes.
- Enable live write-back for the objects that support it, and read SAP's answer.
- Know what stays export-only.

The flow

1. Stewards decide rows in **Work**: accept the suggested value, type a new value, or mark not an issue or accept risk. A change of more than 25 rows, from a key or an **Accept all** button, waits on a confirmation bar with the count: Enter confirms, Escape keeps the rows.
2. Decided values collect in a fix batch, which waits for a second person (**Waiting for approval**). Nobody approves their own batch.
3. An approved batch is sent with **Send to SAP**: the approved values leave as a Migration Cockpit load file that your own process loads. The next run confirms each record.



Work: a change over 25 rows waits on the confirmation bar.

Live write-back

Live write-back is off by default. With `MERIDIAN_LIVE_WRITEBACK_ENABLED=true` in `.env`, the flow is on screen: the inspector of an approved batch shows **Live write-back** (GET `/api/v1/settings` reports `live_writeback: true`). **Check with SAP** lists every field of the batch with BAPI or Export and the reason, and sends nothing; **Send to SAP**, for an approver who did not build the batch, confirms with the record count and shows the result per record with SAP's RETURN messages, and a summary of

records sent, records failed and fields left for the export. The load file stays available as **Export load file**. The same write-back is still open to scripts through the API (`POST /api/v1/writeback`, approved by a second person at `POST /api/v1/writeback/approve/{id}`; the screen uses `POST /api/v1/writeback/batches/{id}/check` and `/send`). Either way it is applied to ECC, S/4HANA on premise or EWM through standard BAPIs:

- **Material master**: `BAPI_MATERIAL_SAVEDATA`.
- **Business partner**: `BAPI_BUPA_CENTRAL_CHANGE` and `BAPI_BUPA_ADDRESS_CHANGE`.
- **Customer** (SD customer master and accounts receivable): `BAPI_CUSTOMER_CHANGEFROMDATA1`.
- **Vendor** (accounts payable), only where the vendor is a business partner with the supplier role: names (`LFA1-NAME1` to `NAME4`) and search term (`SORTL`) by `BAPI_BUPA_CENTRAL_CHANGE`; the address (`LFA1` country, region, city, postal code, language, and `ADRC`) by `BAPI_BUPA_ADDRESS_CHANGE`; bank account holder, control key and reference (`LFBK-KOINH`, `BKONT`, `BKREF`) by `BAPI_BUPA_BANKDETAIL_CHANGE`.

How a vendor is routed is decided on the connected system, per record, before anything is sent: the BAPIs must be there (`RFC_FUNCTION_SEARCH`), `CVI_VEND_LINK` must link the vendor to a business partner (read with `BUT000`), the vendor's address must be one of the partner's (`BUT020`) and a bank row one of its bank details (`BUT0BK`). On S/4HANA every vendor is a business partner. On ECC this is the case only where customer/vendor integration (CVI) is active; there, confirm with the Basis team that synchronisation from the business partner to the vendor is switched on, or a change reaches the business partner and not `LFA1`. **Check with SAP** runs this lookup (reads only) so the field table already says BAPI or Export per vendor. The RFC user needs read access to those tables and `RFC_FUNCTION_SEARCH`, and the write role needs the business partner BAPIs.

For each record Meridian checks the BAPI's interface on the connected system, simulates the call (`TESTRUN`, or a call followed by `BAPI_TRANSACTION_ROLLBACK`; the customer BAPI is simulated by the interface check only, because it may commit by itself), and commits with `BAPI_TRANSACTION_COMMIT` only when SAP's RETURN table holds no `E` or `A` message. Every RETURN message (type, id, number, text) is kept per record in the answer and in `write_back_log`; no field value is logged. A field without a mapping is reported per field with its reason and goes out by export.

Live write-back needs change authorisations, so it uses a **separate** SAP role on a **separate** user, approved by its own process (the template lists the BAPIs and objects). Never add them to the read-only role. Sandbox mode holds every write-back back.

Export only, each field with its reason:

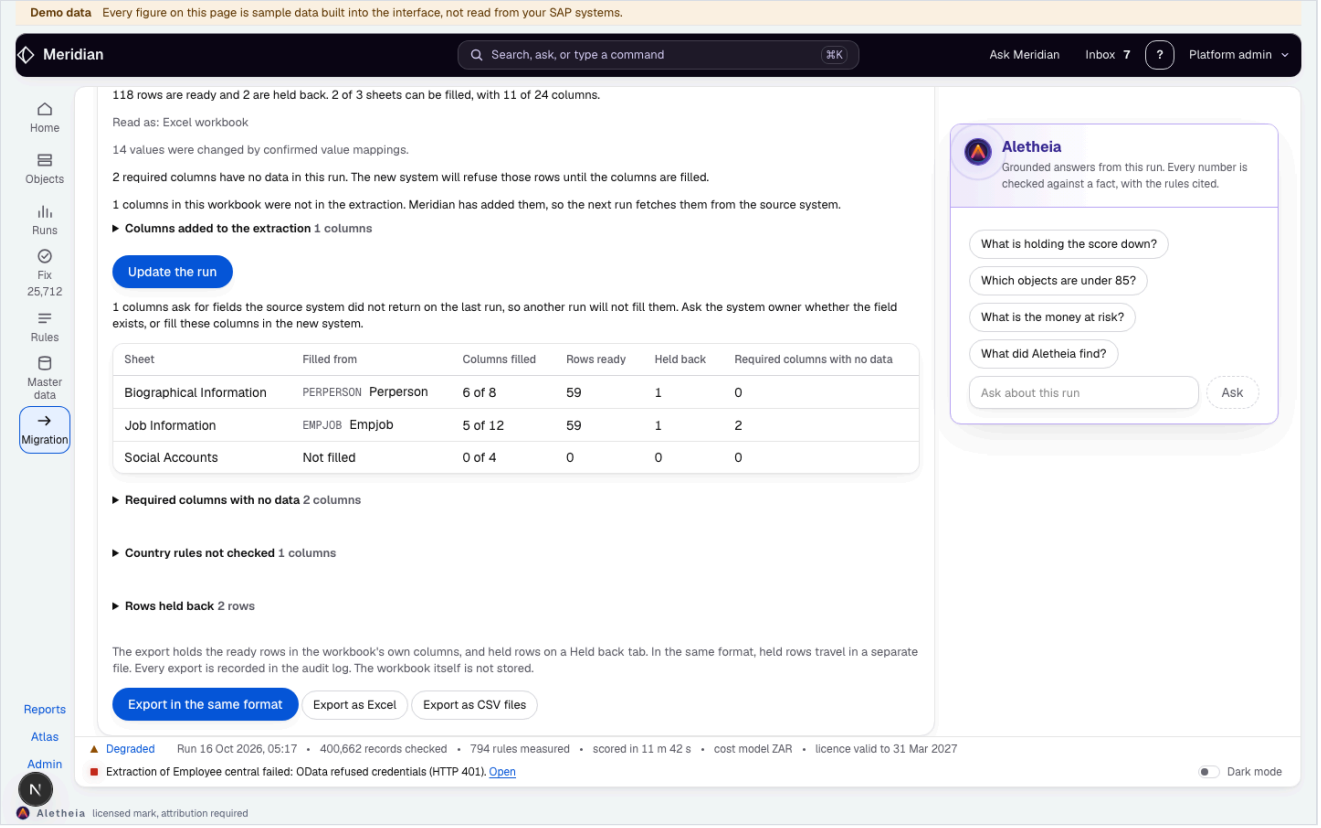
- Vendor company code data (`LFB1`) and purchasing organisation data (`LFM1`): their maintenance (class `VMD_EI_API` and the CVI maintenance APIs) is not remote-enabled, and Meridian relies on no RFC-enabled substitute.
- Every vendor on a release without customer/vendor integration, or not linked to a business partner: the reason reads "Vendor master on this release changes through the vendor maintenance transaction; Meridian exports a load file". SAP has no released remote-enabled change BAPI for the classic vendor master.
- Vendor fields with no mapping (tax numbers, blocks, the bank key itself) and every other object.

Live write-back has not yet been run against a live SAP system by the vendor; run it first on a quality system.

SuccessFactors target workbook

For a SuccessFactors migration, the new system's workbook is the input. On the Migration analysis page, the panel **Workbook from the new system** takes a user with the export permission through three steps: choose a finished run and upload the workbook, **Check the workbook**, then export Excel or CSV in the workbook's own columns. The workbook is read in memory and never stored. Each export is recorded in the audit log as `migration.target_workbook.export`.

The check can change one thing: the extraction. Where a SuccessFactors workbook asks for a column that the extraction does not fetch yet, and the column names a plain property of the sheet's entity in the tenant's own metadata, Meridian adds it to the extraction of that system. The panel counts the added columns and lists them under **Columns added to the extraction**. **Update the run** starts a new extraction and analysis of that system so the columns are fetched now; it needs the permission to start extractions. When the new run finishes, choose it and check the workbook again.



Migration analysis: the workbook from the new system after the check, with the columns added to the extraction and Update the run.

Before a real load, confirm three things with the target system's team:

- Key columns. Definition sheets leave out person and user ids, so Meridian adds them under source property names (`personIdExternal`, `userId`). The import template may spell them differently.
- Required columns with no data. Meridian fills only what the run extracted. It leaves other columns empty and does not hold those rows back, so the check lists them and the new system refuses those rows until they are filled.

- Lists of values. A workbook with no list values cannot check those columns.

Checks

- With write-back off, an approval answers that live write-back is off on this deployment and points to the export.
- With it on, a record SAP refuses reads "SAP refused the change; see its RETURN messages" and nothing is committed.

Exercise

On a quality system, with live write-back on, open an approved batch of material master fixes, run **Check with SAP**, then **Send to SAP** as a second person, and read the simulation messages and the commit messages for each record. Then open an approved accounts payable batch: run **Check with SAP** and read which vendor fields go by BAPI (a vendor that is a business partner) and which by export, and why (a company code field always does).

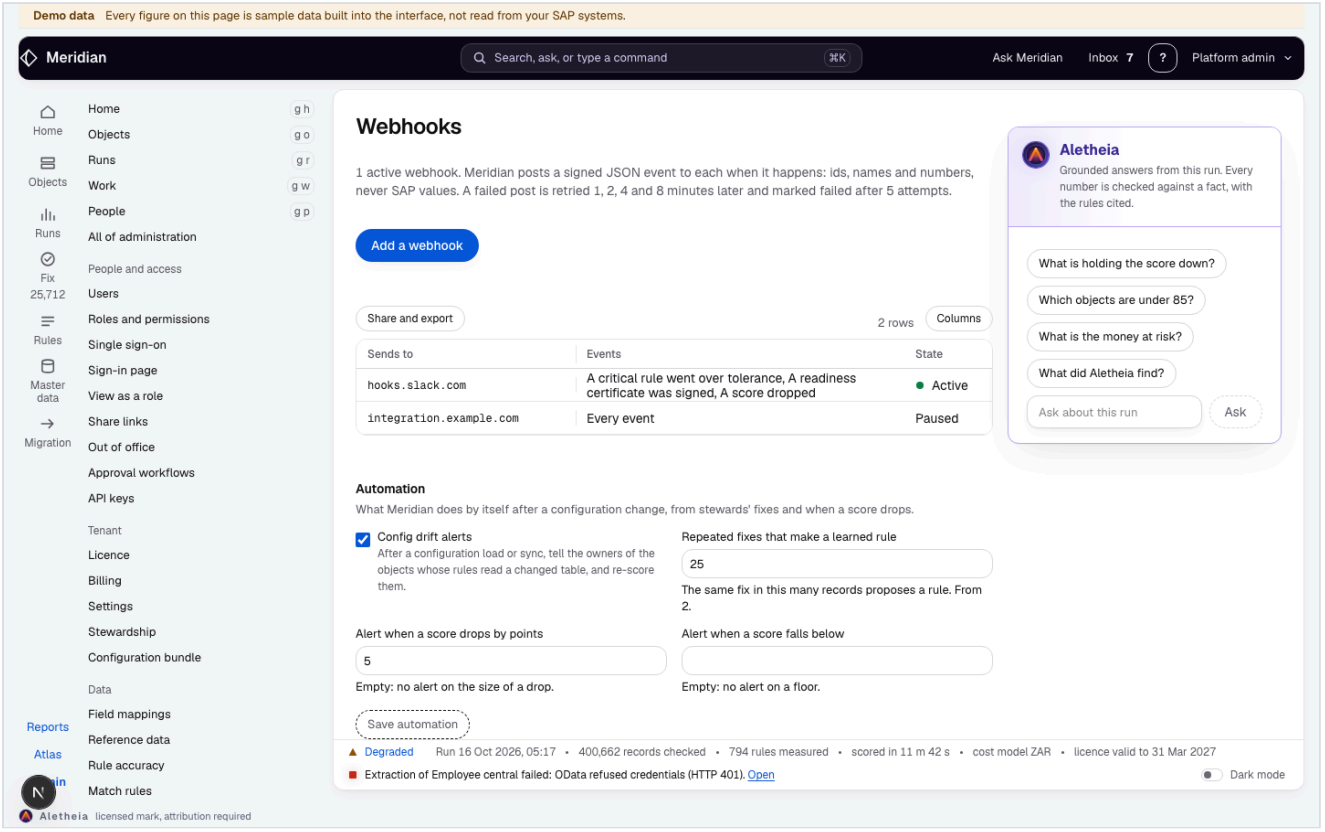
Module 8: Notifications and integrations

Learning objectives

- Set the tenant's default digest and scheduled digests.
- Add webhooks for Slack, Teams or any https endpoint.

Steps

1. **Administration, Alerts**: channels (email, Microsoft Teams, signed webhook), **When to alert** and **Score floors per object**. Alerts carry counts, rule ids and links, never record values.
2. **Administration, Default digest**: the summary the tenant gets by default. Each person sets their own digest under Preferences.
3. **Administration, Digests**: scheduled summaries for a role or a migration wave, with cadence, time zone and channel, one row each showing audience, cadence, channel, when it was last sent and when it runs next, and On or Paused; **Preview**, **Send now**, **Pause** and the delivery log. Email needs the `SMTP_*` or Microsoft Graph settings; without them an email digest is recorded as failed with the reason.
4. **Administration, Webhooks**: **Add a webhook** with the URL of a Slack or Teams workflow, or any https endpoint, and the events it should receive. Meridian posts a signed JSON event (ids, names and numbers, never SAP values); a failed post is retried after 1, 2, 4 and 8 minutes and marked failed after 5 attempts.
5. Set `MERIDIAN_APP_URL` in `.env` so links in digests, webhooks and invitations open your deployment.



Administration, Webhooks: signed events and the delivery log.

Checks

- You should now see the webhook as **Active**, and a test delivery in its log.

Exercise

Create a Teams workflow that receives a webhook, add it for the `digest.sent` event, and send a digest now.

Module 9: Operations

Learning objectives

- Back up and restore, with the `.env` encrypted.
- Update, pin a version and roll back.
- Know what watches the platform and the data after a clean.
- Know log rotation, DNS, the diagnostics bundle, the status page and sandbox mode.
- Read the Aletheia engine page and set the engine up in `docker/.env`.

Backups

`scripts/backup.sh` writes `backups/<timestamp>/` with the database dump, the object storage, a manifest, and the `.env` file encrypted with GPG (`env.gpg`), because `.env` holds the master keys. It prompts for the passphrase; for cron, give it a root-only passphrase file:

FENCE0 With no terminal and no passphrase file the backup stops rather than writing `.env` in clear. `--plain-env` opts out (writes `env.txt`, mode 600). `MERIDIAN_BACKUP_PASSPHRASE_FILE` can stand in for the flag.

Restore with `sudo bash scripts/restore.sh <backup-dir>`; it decrypts `env.gpg` with `MERIDIAN_BACKUP_PASSPHRASE_FILE` when set, else it prompts. `--drill` restores into a temporary database without touching the real one: run it monthly.

Updates and rollback

- **Administration, Platform and updates** shows the running and the latest version, and whether the updater is set up on this deployment. When a newer version is available and the updater is set up, **Update to** that version, confirmed with **Run the update**, starts it; progress shows on the same card. Running an update needs the manage system permission. Without the updater the card says so: run the update script on the host instead.
- On the host: - `sudo bash scripts/update.sh` updates to the latest release that Meridian HQ names. When HQ cannot be reached, nothing is changed and the script asks for `--version`. - `sudo bash scripts/update.sh --version 2.4.1` updates to, or reinstalls, one release. - `sudo bash scripts/update.sh --rollback` returns to the release that ran before. A second `--rollback` returns to the newer one again.
- An update runs in this order: it snapshots the current images as `:rollback`, pulls the new release, keeps HTTPS on in nginx, writes a dump of the whole database to `backups/pre-update-<time>.dump`, pins `MERIDIAN_VERSION` to the new release in `.env` (the replaced one is kept as `MERIDIAN_PREVIOUS_VERSION`), runs the database migrations and probes `/health` for up to 300 seconds. If the database dump fails, the update stops and nothing is changed.
- If the migrations or the health probe fail, the script rolls back to the previous images by itself and prints the `pg_restore` command for the pre-update dump. Rollback restores images, not the schema: when the migrations had already run, restore the dump as printed.
- `--no-verify` skips the health probe. `--include-updater` also upgrades the updater's own image as the very last step; the in-product update never passes it.

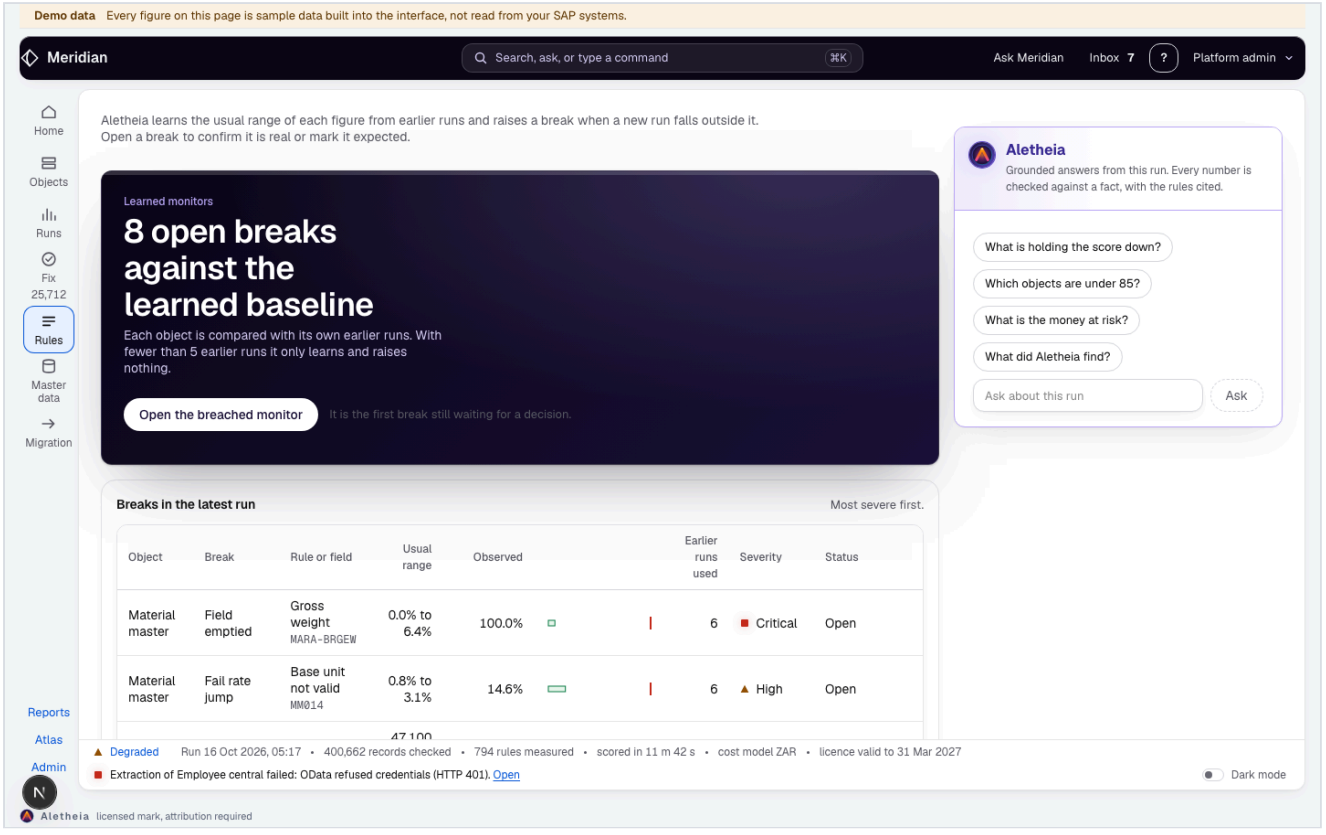
- `MERIDIAN_VERSION` in `.env` sets the Meridian image tag; a fixed tag holds the stack at that release.

Logs, DNS and images

- Every container rotates its logs: at most 5 files of 50 MB each.
- Redis is pinned to `redis:7.2-alpine` (7.4 and later are not open source) and Ollama to a fixed release.
- DNS: the host's resolver by default; `MERIDIAN_DNS_1` and `MERIDIAN_DNS_2` override it (module 4).

Monitoring

- **Administration, Platform and updates, Health checks** and the status page (below) watch the platform itself.
- **Administration, Alerts** (module 8) sends what you choose under **When to alert** and the **Score floors per object** to email, Teams or a webhook.
- After a cleaning batch is loaded, **Monitoring** under Work shows whether the fixed records stay fixed and which rules pull them back. It needs a run after the load; until then it says there is nothing to monitor yet.
- **Learned monitors**, under Rules, compares the latest run of each object with that object's own earlier runs and needs no setup. Between full runs it also compares the rows each sync extracted with the earlier syncs.

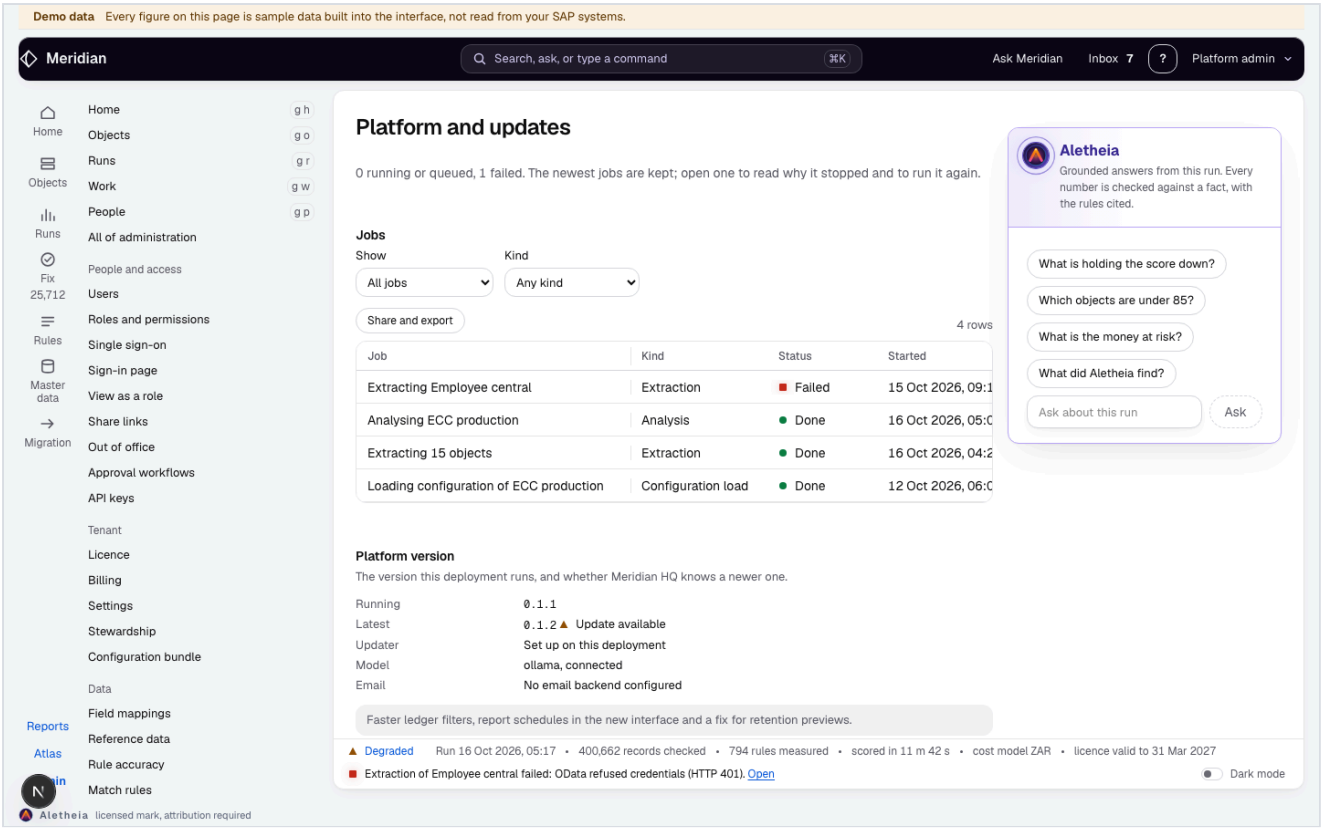


Learned monitors: breaks found against each object's own earlier runs.

Support and safety

- **Download diagnostics bundle** under Administration, Platform and updates: versions, health checks, recent errors and configuration with every secret removed. It holds no SAP record values.

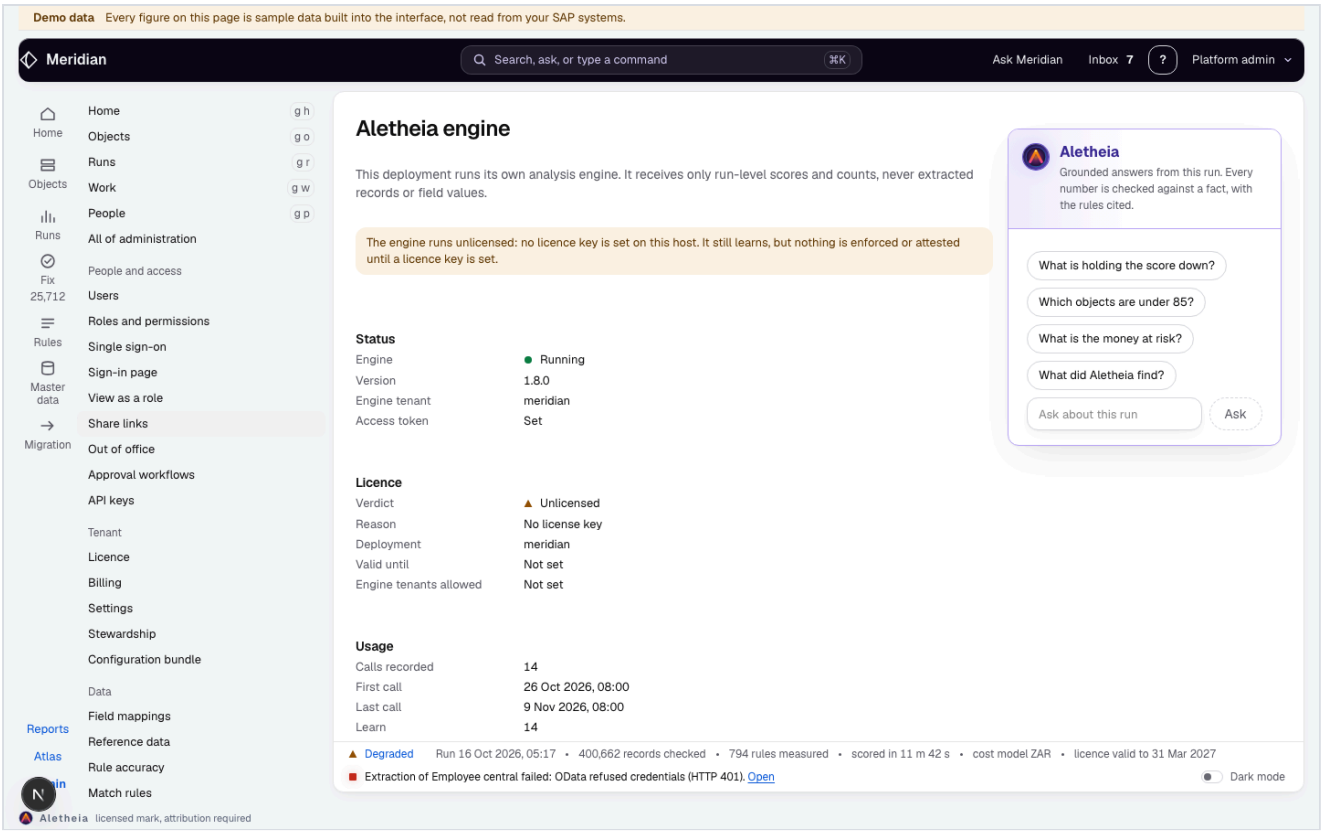
- **The status page** at `/status` is readable signed out.
- **Sandbox mode** (a switch under Administration, Settings): copies the latest run; decisions land on the copy and notifications, webhooks, digests, report mailouts and write-backs are held back until you exit.



Administration, Platform and updates: jobs, version, health checks and the diagnostics bundle.

Aletheia engine

Administration, Aletheia engine (under Operations, with the manage system permission) shows the deployment's own analysis engine. The engine runs as a separate container next to Meridian, with its own volume, and is never shared with another product on the same host. It receives only run-level scores and counts, never extracted records or field values. When the engine is missing or stopped, Meridian keeps working; only the engine panels stay empty.



Administration, Aletheia engine: status, licence and use of the deployment's own engine.

The page has three blocks.

- **Status** - **Engine**: *Running*, *Not answering*, or *Not configured* when no engine address is set. - **Version**: the engine release that answered. - **Engine tenant**: the tenant Meridian learns into inside the engine (`meridian` unless changed). - **Access token**: *Set* or *Not set*. The token itself is never shown.
- **Licence** - **Verdict**: *Valid*, *Not valid*, *Unlicensed*, *Not read* (the engine answered but gave no licence) or *Not known* (the engine does not answer). - **Reason**: why the licence is not valid, or *None*. - **Deployment**: the deployment id the licence was issued for. - **Valid until**: the licence end date. - **Engine tenants allowed**: how many engine tenants the licence permits.
- **Usage** - **Calls recorded**, **First call** and **Last call**: how often and when Meridian has used the engine. - One further row per call type, with its count.

When something is wrong, a note above the blocks names the first problem found, checked in this order:

1. **Not configured**: no engine address is set, so Meridian runs without its engine.
2. **Not answering**: the address is set but the engine does not respond. Check that the `aletheia` container runs.
3. **No access token**: the engine runs, but Meridian cannot send it run trends to learn from.
4. **Unlicensed**: no licence key is set on this host. The engine still learns, but nothing is enforced or attested.
5. **Licence not valid**: the note gives the reason. Meridian stops sending the engine work until a valid key is set.

The engine is set up in `docker/.env` with four settings. Keep the values in `.env` only (it is encrypted in every backup); never paste them into tickets, chat or screenshots.

FENCE1 - `ALETHEIA_DEPLOYMENT_ID` must match the id on the licence, or the verdict is *Not valid*. It defaults to `meridian`. - `ALETHEIA_LICENSE_KEY` empty means *Unlicensed*. - `ALETHEIA_TOKEN` is given to both Meridian and the engine container; empty means *Access token: Not set*. Generate one with `openssl rand -hex 32`. - `MERIDIAN_ALETHEIA_IMAGE` is normally left unset, so the engine release that ships with this Meridian release is used.

After changing any of them, run `sudo bash scripts/update.sh --version <the running version>` (it starts the engine by name) and reload the page.

Checks

- `backups/<timestamp>/env.gpg` should exist and `env.txt` should not.
- `sudo bash scripts/restore.sh --drill <backup-dir>` should exit 0.

Exercise

Create the passphrase file, schedule the nightly backup in cron, run one restore drill, and record the time it took as your restore time.

Module 10: Troubleshooting

Symptom	Cause	Fix
`http://` works but `https://` refuses	SSL mode 1 was chosen, or no certificate in `nginx/certs`	Re-run the installer with SSL mode 2 or 3; `update.sh` keeps HTTPS on afterwards
Sign-in goes straight to a password change	The account must change its password (first start or admin reset)	Set a password of 12 or more characters that is not the email name or a common one
"10 failed sign-ins lock the account for 15 minutes"	Lockout after failed sign-ins (`auth.lockout` in the audit log)	Wait 15 minutes, or reset the password under Users
`/health` answers 503 `degraded`	Postgres or Redis does not answer	`docker compose ps`, then the logs of `db` or `redis`
`/docs` answers 404	API docs are hidden outside development	Set `MERIDIAN_OPENAPI_PUBLIC=1` only where nginx is not public
"Licence needed": Meridian HQ could not be reached	The licence server was unreachable past the offline grace (`licence_server_cutoff`)	Restore outbound HTTPS to the licence server; check `X-Licence-Warning` and `offline_grace` days earlier
A licence is refused after an update	Legacy signature with the transition window off	Set `MERIDIAN_LICENCE_ALLOW_LEGACY_SIGNATURES=true` until HQ issues an Ed25519-signed licence
Connection test: "Sign-in refused" or HTTP 401	Wrong or expired user or secret	Connection settings: type the new values and test again
"No authorisation to read this table"	The RFC user lacks `S_TABU_NAM` or `S_TABU_DIS` on that table	Add it from the role template; SU53 shows the check
Extraction refused at the reader	The chosen reader is not authorised or does not exist	Set `MERIDIAN_RFC_READ_FUNCTION` to a reader your role grants, or `auto`
SAP host names do not resolve	Containers use the host resolver	Set `MERIDIAN_DNS_1/2` in `.env` and re-run `update.sh`
A schedule did not run	Due runs start within 5 minutes; workers down	Runs, Jobs; the background workers health check

Symptom	Cause	Fix
Amounts read "default estimate"	No tenant cost model saved	Administration, Settings, Cost model and currency
Every object reads 70.0	Two or more critical rules over tolerance cap each one	Read the measured score beside it; fix the named rules
Approval: "Live SAP write-back is off on this deployment"	`MERIDIAN_LIVE_WRITEBACK_ENABLED` is off	Deliver by export, or enable it on a tested deployment
Backup stops: "no terminal to ask for the .env passphrase"	Cron without a passphrase file	`--passphrase-file` or `MERIDIAN_BACKUP_PASSPHRASE_FILE`
No email arrives	No email backend configured	`SMTP_*` or Microsoft Graph settings; Health checks, Email delivery
"LLM connection failed" in `/health`	The model endpoint is unreachable	Check `OLLAMA_BASE_URL` and the Ollama container; scores never depend on it
Services will not start	Disk, memory or ports	`df -h`, `free -h`, `docker compose logs`

For anything else, download the diagnostics bundle and send it with your request: it holds no SAP data, no credentials and no user emails. The printable user guide (all roles) is `frontend/docs/guide/meridian-guide.pdf`.

Module 11: Prove the numbers

You will learn to reproduce a rule's count by hand in SAP, so your SAP team can check Meridian's numbers without trusting Meridian, and record the result for the go-live sign-off.

Every count in Meridian is a rule over the data your extraction read. For any rule and run, Meridian gives the SE16 selection that reproduces it: the table and record grain, the selection fields with their SE16 options and values, what SE16 alone cannot express (a pattern, a check-table lookup, a duplicate, a join) in plain words, the population and failing counts it measured, and five failing record keys as SE16 shows them.

Steps

1. Open a rule (⌘K, then its id, for example `MM014`) and choose **Prove in SAP**. When the rule runs on several objects, choose the object and run.
2. In SAP, open the transaction the section names: `SE16N` when one selection is enough, otherwise `SE16H` (or `SQVI` for a join). Enter the table of the **Record grain**.
3. Enter the **Population** rows. "Initial value" means option `=` with an empty value (it selects blank, 0 and 00000000). The population leaves out records SAP marks as dead, such as `LVORM` flagged for deletion. Count the entries and compare with **Population**.
4. Add the **Failing** rows and count again; compare with **Failing**. For a rule with conditions SE16 cannot express, apply them as written (for a check table, list the check table in SE16 and compare the values).
5. Look up the **Five failing records** by their key. SE16 shows `MATNR 232`; Meridian stores `MATNR=0000000000000232` (the conversion exit adds the leading zeros), and both are shown.

To prove a whole run, **Download the reconciliation workbook** from the same section (export permission): a summary sheet with one row per rule, for the ten rules with the most failing records, and one sheet per rule with its selection, steps, counts and sample keys. Your team fills in the **SE16 count** and **Match** columns.

Check. The SE16 count equals Meridian's count when the extraction read the table in full on the run's date. A small difference usually means the data changed since the extraction (SE16 reads today's data); rerun the extraction and the rule, then count again in the same hour. A table read by a window (an extraction scope or a date range) is counted only inside that window: add the same window to the SE16 selection.

Exercise. Reconcile three rules of your first run: one completeness rule with a single SE16N selection (a `null_check`), one value-list rule (a check-table lookup) and one rule with a join. Fill in the workbook and keep it with the go-live records. The step-by-step first deployment runbook is `docs/live-deployment-checklist.md`.